Tutorial — Proyecto Final — Dashboard de Ventas Retail con Streamlit

1. Información del Proyecto

Módulo: 5 — Proyecto Final
Tema: Dashboard de Ventas Retail con Login, KPIs y Mapa Geográfico
Dataset: Retail Sales Dataset (Kaggle — Abbas829)
Tecnologías: Python · Streamlit · Pandas · Plotly
Nivel: Principiante — no se requiere experiencia previa con aplicaciones web

2. ¿Qué vamos a construir?

Vamos a construir **RetailDash**: una app web de análisis de ventas retail que cualquier persona puede usar desde el navegador. La app tiene:

- Una **pantalla de login** donde el usuario ingresa su usuario y contraseña
- Un **dashboard con 4 páginas**: resumen, ventas, mapa y explorador de datos
- **6 KPIs** (indicadores clave) calculados automáticamente del CSV
- **Gráficas interactivas** con Plotly (líneas, barras, donut, scatter, treemap)
- Un **mapa de EE.UU.** con burbujas proporcionales a las ventas por región
- **Filtros** en la barra lateral para explorar los datos por categoría, región y año

Todo esto con **Python puro** — sin HTML, sin JavaScript, sin servidores. Solo Streamlit.

3. Descripción del Dataset

El dataset de Kaggle contiene registros de ventas diarias de una tienda retail simulada. Tiene las siguientes columnas:

Columna	Tipo	Descripción
`Date`	Fecha	Fecha del registro de venta
`Category`	Texto	Categoría del producto. ⚠️ Tiene valores `"NaN?"` que hay que limpiar
`Sales`	Número	Ingresos totales de ventas en USD
`Quantity`	Número	Cantidad de productos vendidos
`Profit`	Número	Ganancia neta generada
`Region`	Texto	Región geográfica donde ocurrió la venta

Problema de calidad de datos: La columna `Category` contiene entradas inválidas como `"NaN?"` (literalmente la cadena de texto «NaN?», no un valor nulo real). Nuestro código de limpieza se encarga de eliminarlas automáticamente.

4. Estructura del Proyecto

Antes de escribir una sola línea de código, es importante entender cómo se organiza el proyecto. Cada archivo tiene una responsabilidad clara:

```
retaildash/              <- carpeta raíz del proyecto
|
+-- app.py               <- PUNTO DE ENTRADA: el único archivo que ejecutamos
+-- auth.py              <- LOGIN: pantalla de inicio de sesión y validación
+-- data_loader.py       <- DATOS: carga el CSV y lo limpia
+-- utils.py             <- UTILIDADES: funciones compartidas por todas las páginas
+-- requirements.txt     <- DEPENDENCIAS: lista de librerías necesarias
|
+-- pages/               <- PÁGINAS DEL DASHBOARD (Streamlit las detecta
automáticamente)
|   +-- 1_Resumen.py     <- Página 1: KPIs + gráficas de tendencia temporal
|   +-- 2_Ventas.py      <- Página 2: análisis detallado por categoría y región
|   +-- 3_Geografico.py  <- Página 3: mapa interactivo + tabla por región
|   +-- 4_Datos.py       <- Página 4: explorador de datos + descarga CSV
|
+-- data/
    +-- retail_sales.csv <- El CSV de Kaggle va aquí
```

¿Por qué tantos archivos? Porque separar el código en módulos hace que sea más fácil de leer, mantener y explicar. Si algo falla, sabes exactamente en qué archivo buscar.

5. Cheat Sheet — Referencia Rápida de Streamlit

Antes de empezar a codificar, aquí tienes los comandos más importantes que vas a usar en este proyecto. Úsalo como referencia mientras trabajas.

5.1. Configuración y arranque

Código	¿Qué hace?
`st.set_page_config(page_title="...", layout="wide")`	Configura el título de la pestaña del navegador y el ancho de la página. Solo se llama **una vez** al inicio de `app.py`
`streamlit run app.py`	Comando de terminal para arrancar la app. Abre el navegador en `http://localhost:8501`
`st.rerun()`	Fuerza que el script se vuelva a ejecutar desde cero. Útil después de cambiar `session_state`
`st.stop()`	Detiene la ejecución del script en ese punto. Todo lo que venga después no se ejecuta

5.2. Textos y estructura visual

Código	¿Qué hace?
`st.title("texto")`	Título grande, el más importante de la página
`st.subheader("texto")`	Subtítulo de sección
`st.caption("texto")`	Texto pequeño en gris, para notas secundarias
`st.write("texto")`	Texto normal, también acepta variables Python

Código	¿Qué hace?
`st.divider()`	Línea horizontal separadora
`st.info("texto")`	Caja azul informativa
`st.success("texto")`	Caja verde de éxito
`st.warning("texto")`	Caja amarilla de advertencia
`st.error("texto")`	Caja roja de error

5.3. Layout (organización visual)

Código	¿Qué hace?
`col1, col2 = st.columns(2)`	Crea 2 columnas de igual ancho
`col1, col2 = st.columns([1, 2])`	Crea 2 columnas: la segunda es el doble de ancha
`with col1: st.write("...")`	Todo lo que esté dentro del `with` aparece en esa columna
`_, col, _ = st.columns([1, 1.5, 1])`	Truco para centrar: columna central más ancha, las laterales ignoradas con _
`with st.sidebar: st.write("...")`	Muestra contenido en la barra lateral izquierda
`with st.expander("ver más"): st.write("...")`	Contenido colapsable que el usuario puede expandir

5.4. Widgets de entrada (inputs)

Código	¿Qué hace?
`st.text_input("label")`	Caja de texto de una línea. Devuelve un `str`
`st.text_input("label", type="password")`	Igual pero oculta los caracteres con ●●●
`st.selectbox("label", opciones)`	Menú desplegable. Devuelve la opción elegida
`st.multiselect("label", opciones, default=[...])`	Selección múltiple con checkboxes. Devuelve lista
`st.button("texto", type="primary")`	Botón. Devuelve `True` solo en el click
`st.download_button("texto", data=csv, ...)`	Botón especial que descarga un archivo

5.5. Formularios

Código	¿Qué hace?
`with st.form("nombre_unico"):`	Agrupa widgets. La app no recarga hasta hacer submit
`st.form_submit_button("Enviar")`	Botón de submit dentro de un form. Devuelve `True` al hacer click

¿Cuándo usar `st.form`? Siempre que tengas varios inputs relacionados (como usuario + contraseña). Sin form, la app recarga al escribir cada letra. Con form, solo recarga al hacer clic en «Enviar».

5.6. Visualización de datos

Código	¿Qué hace?
`st.metric("label", valor)`	Tarjeta KPI con número grande
`st.metric("label", valor, delta="+5%")`	KPI con variación (verde si positiva, rojo si negativa)
`st.dataframe(df)`	Tabla interactiva del DataFrame
`st.plotly_chart(fig, use_container_width=True)`	Muestra una gráfica de Plotly a ancho completo

5.7. Session State

Código	¿Qué hace?
`st.session_state`	Diccionario que persiste entre re-ejecuciones
`if "clave" not in st.session_state:`	Verifica si una clave ya existe (para inicializarla solo una vez)
`st.session_state.logged_in = True`	Guarda un valor (notación de punto)
`st.session_state["logged_in"] = True`	Guarda un valor (notación de diccionario, igual que la anterior)
`st.session_state.get("logged_in", False)`	Lee un valor con valor por defecto si no existe

5.8. Cache

Código	¿Qué hace?
`@st.cache_data`	Decorador: la función se ejecuta solo la primera vez. El resultado se guarda en memoria
`@st.cache_data(show_spinner="Cargando...")`	Igual, pero muestra un mensaje mientras carga la primera vez

5.9. Navegación entre páginas

Código	¿Qué hace?
`st.Page("archivo.py", title="Nombre", icon="📊")`	Define una página del dashboard
`st.navigation([pagina1, pagina2, ...])`	Crea la barra de navegación con las páginas dadas
`pg = st.navigation(...); pg.run()`	Ejecuta la página actualmente seleccionada
`position="hidden"`	Parámetro de `st.navigation` para ocultar la barra lateral (útil en login)

6. Construcción del Proyecto — Paso a Paso

Paso 1 — Instalación y preparación del entorno

*Este es el único paso que se hace **fuera de Python**. Todo se hace desde la terminal (cmd en Windows, Terminal en Mac/Linux).*

***¿Qué es una terminal?** Es la ventana de texto donde escribes comandos.*

Paso 1 — Crea la carpeta del proyecto:

```
mkdir retaildash
cd retaildash
mkdir pages data
```

- `mkdir` *significa «make directory» (crear carpeta)*
- `cd` *significa «change directory» (entrar a la carpeta)*
- *Después de estos comandos, tu carpeta raíz es* `retaildash/`

Paso 2 — Instala las librerías necesarias:

```
pip install streamlit pandas numpy plotly
```

Esto puede tardar 1-2 minutos. Verás mucho texto en pantalla — es normal. Cuando termine verás `Successfully installed ...`.

- `streamlit` *-> el framework que convierte Python en una app web*
- `pandas` *-> para manipular tablas de datos (DataFrames)*
- `numpy` *-> para cálculos matemáticos (lo usa pandas internamente)*
- `plotly` *-> para crear gráficas interactivas*

Paso 3 — Descarga el dataset de Kaggle:

Entra a este enlace: https://www.kaggle.com/datasets/abbas829/retail-sales-dataset

Necesitas una cuenta en Kaggle (es gratis). Haz clic en «Download» y mueve el archivo CSV descargado a la carpeta `retaildash/data/`*. Renómbralo a* `retail_sales.csv` *si tiene otro nombre.*

Paso 4 — Verifica que Streamlit funciona:

```
streamlit hello
```

Si se abre una ventana en tu navegador con una app de demostración, ¡Streamlit está instalado correctamente! Ciérrala con `Ctrl+C` *en la terminal.*

Paso 5 — Crea el archivo `requirements.txt`:

Crea un archivo de texto llamado `requirements.txt` *en la carpeta raíz con este contenido:*

```
streamlit>=1.32.0
pandas>=2.0.0
numpy>=1.26.0
plotly>=5.18.0
```

Este archivo sirve para que otros puedan instalar exactamente las mismas versiones con un solo comando: `pip install -r requirements.txt`.

Paso 2 — Crear `app.py` — El punto de entrada de toda la aplicación

*`app.py` es el **director de orquesta**. No muestra datos ni gráficas por sí mismo — su único trabajo es decidir qué mostrar: ¿la pantalla de login o el dashboard?*

Crea el archivo `retaildash/app.py` con este contenido:

```python
import streamlit as st
from auth import show_login

# -- 1. Configuración global ---------------------------------------
# set_page_config SIEMPRE debe ser la primera llamada a Streamlit.
# Si lo pones en otro archivo o después de otro comando, da error.
st.set_page_config(
    page_title="RetailDash",
    page_icon="📈",   # aparece en la pestaña del navegador
    layout="wide",    # usa todo el ancho de la pantalla
)

# -- 2. Inicialización de session_state ----------------------------
# Streamlit re-ejecuta el script completo en cada interacción.
# session_state es el único lugar donde los datos "sobreviven"
# entre re-ejecuciones. Lo inicializamos solo si aún no existe.
if "logged_in" not in st.session_state:
    st.session_state.logged_in = False   # ¿está logueado? No.
    st.session_state.username  = ""      # nombre de usuario vacío
    st.session_state.nombre    = ""      # nombre completo vacío
    st.session_state.rol       = ""      # rol vacío

# -- 3. Definición de las páginas del dashboard --------------------
# st.Page() vincula un archivo Python con un título e ícono.
# Streamlit mostrará estas páginas en la barra lateral.
dashboard_pages = [
    st.Page("pages/1_Resumen.py",     title="Resumen",     icon="📊", default=True),
    st.Page("pages/2_Ventas.py",      title="Ventas",      icon="💰"),
    st.Page("pages/3_Geografico.py", title="Geográfico", icon="🗺️"),
    st.Page("pages/4_Datos.py",       title="Datos",       icon="📁"),
]

# -- 4. Routing: ¿mostrar login o dashboard? ----------------------
if not st.session_state.logged_in:
    # El usuario NO está logueado -> mostrar login
    # position="hidden" oculta la barra lateral para no distraer
    pg = st.navigation(
        [st.Page(show_login, title="Login")],
        position="hidden",
    )
    pg.run()
else:
    # El usuario SÍ está logueado -> mostrar el dashboard
    pg = st.navigation(dashboard_pages)
    pg.run()
```

¿Qué es `session_state` y por qué lo necesitamos?

Streamlit funciona diferente a un programa normal. Cada vez que el usuario hace clic en un botón o escribe algo, Python ***vuelve a ejecutar el script completo de arriba a abajo****. Si guardáramos `logged_in = True` en una variable normal, en la próxima ejecución volvería a ser `False`.*

`session_state` es como una «memoria» especial que Streamlit mantiene entre ejecuciones. Es un diccionario que persiste mientras el usuario tiene la app abierta en el navegador.

Analogía: *Imagina que cada vez que el usuario interactúa con la app, le das una hoja de papel nueva para escribir. `session_state` es como una pizarra que no se borra — siempre está ahí.*

Paso 3 — Crear `auth.py` — Sistema de Login

Este archivo maneja todo lo relacionado con la autenticación: los usuarios registrados y la pantalla de login.

Crea el archivo `retaildash/auth.py`:

```python
import streamlit as st

# -- Diccionario de usuarios ---------------------------------------
# En una aplicación real, esto estaría en una base de datos
# con contraseñas encriptadas (hashed). Para esta demo,
# usamos un diccionario de Python por simplicidad.
#
# Estructura: { "usuario": { "password": "...", "nombre": "...", "rol": "..." } }
USUARIOS = {
    "admin": {
        "password": "admin123",
        "nombre":   "Administrador",
        "rol":      "Admin",
    },
    "ana": {
        "password": "ana2024",
        "nombre":   "Ana García",
        "rol":      "Analista",
    },
    "carlos": {
        "password": "carlos2024",
        "nombre":   "Carlos López",
        "rol":      "Viewer",
    },
}


def verificar_credenciales(usuario: str, password: str) -> bool:
    """
    Verifica si el usuario y contraseña son correctos.
    Devuelve True si son válidos, False si no.
```

```
    Los : str y -> bool son "type hints" (pistas de tipo).
    No son obligatorios, pero ayudan a entender qué recibe
    y qué devuelve la función.
    """
    if usuario in USUARIOS:                             # ¿existe el usuario?
        return USUARIOS[usuario]["password"] == password  # ¿coincide la contraseña?
    return False  # el usuario no existe -> False


def show_login():
    """
    Dibuja la pantalla de login.
    Esta función es llamada desde app.py cuando el usuario no está logueado.
    """

    # Truco para centrar el formulario:
    # Creamos 3 columnas con proporciones [1, 1.5, 1]
    # La columna del medio es la más ancha -> ahí va el formulario
    # El guión bajo (_) es una convención para decir "no uso esta variable"
    _, col, _ = st.columns([1, 1.5, 1])

    with col:
        st.title("🛒 RetailDash")
        st.subheader("Iniciar sesión")
        st.caption("Dashboard de Ventas · Módulo 5")
        st.divider()

        # -- st.form ------------------------------------------------
        # Sin st.form: Streamlit recarga el script cada vez que
        # el usuario escribe una letra. Con st.form: solo recarga
        # cuando el usuario hace clic en "Iniciar sesión".
        # El string "login_form" es un ID único para este formulario.
        with st.form("login_form"):
            usuario  = st.text_input("👤 Usuario",    placeholder="Ej: admin")
            password = st.text_input(
                "🔒 Contraseña",
                type="password",   # muestra ●●●● en vez del texto
            )
            submit = st.form_submit_button(
                "Iniciar sesión ->",
                use_container_width=True,  # botón ocupa todo el ancho
                type="primary",            # botón en color rojo de Streamlit
            )

        # -- Lógica de validación -----------------------------------
        # Este bloque se ejecuta DESPUÉS del form (fuera del with)
        # Solo hace algo cuando submit es True (cuando el usuario hizo clic)
        if submit:
            if not usuario or not password:
                # .strip() elimina espacios al inicio y al final
                st.warning("⚠️ Completa todos los campos.")

            elif verificar_credenciales(usuario.strip().lower(), password):
```

```
            # ¡Credenciales correctas! Guardamos todo en session_state
            # .lower() convierte a minúsculas para evitar problemas
            # con "Admin" vs "admin" vs "ADMIN"
            u = usuario.strip().lower()
            st.session_state.logged_in = True
            st.session_state.username  = u
            st.session_state.nombre    = USUARIOS[u]["nombre"]
            st.session_state.rol       = USUARIOS[u]["rol"]

            st.success("✅ ¡Bienvenido/a! Cargando dashboard...")

            # st.rerun() fuerza que el script se vuelva a ejecutar.
            # Como ahora logged_in=True, app.py mostrará el dashboard.
            st.rerun()

        else:
            st.error("❌ Usuario o contraseña incorrectos.")

    st.divider()
    # Mostramos las credenciales de demo para que los alumnos puedan probar
    st.info("🔑 **Demo:** admin/admin123 · ana/ana2024 · carlos/carlos2024")
```

¿Por qué `usuario.strip().lower()`?

- *`.strip()` elimina espacios accidentales: `" admin "` -> `"admin"`*
- *`.lower()` convierte a minúsculas: `"Admin"` -> `"admin"`*

Esto evita que el usuario se quede sin poder entrar porque escribió `"Admin"` con mayúscula cuando en el diccionario está `"admin"`.

¿Por qué `st.form_submit_button` en vez de `st.button`?

`st.button` normal haría que la app recargara cada vez que el usuario escribe en los campos de texto. `st.form_submit_button` dentro de un `st.form` agrupa todo: la app espera a que el usuario haga clic en el botón para recargar.

Paso 4 — Crear `data_loader.py` — Carga y Limpieza del CSV

Este módulo tiene una sola responsabilidad: leer el CSV de Kaggle, limpiarlo y devolver un DataFrame listo para analizar.

Crea `retaildash/data_loader.py`:

```
import pandas as pd
import numpy as np
import streamlit as st


# -- Coordenadas geográficas por región ----------------------------
# El dataset no tiene coordenadas GPS, así que las definimos
# manualmente. Estas son posiciones aproximadas de cada región
# dentro de EE.UU., usadas para dibujar el mapa.
REGION_COORDS = {
```

```
    "North":   {"lat": 43.0, "lon": -85.0},  # zona norte (Michigan)
    "South":   {"lat": 30.0, "lon": -90.0},  # zona sur (Louisiana)
    "East":    {"lat": 40.7, "lon": -74.0},  # zona este (New York)
    "West":    {"lat": 37.8, "lon": -120.5}, # zona oeste (California)
    "Central": {"lat": 41.8, "lon": -93.0},  # zona central (Iowa)
}


# -- @st.cache_data ------------------------------------------------
# Este decorador le dice a Streamlit: "ejecuta esta función solo
# la primera vez. Guarda el resultado en memoria y devuélvelo
# en las siguientes llamadas sin volver a ejecutarla."
#
# Sin caché: cada vez que el usuario mueve un filtro, Python
# vuelve a leer el CSV completo -> muy lento.
# Con caché: lee el CSV una vez, guarda el resultado,
# las siguientes llamadas son instantáneas.
#
# show_spinner muestra un mensaje de carga la primera vez.
@st.cache_data(show_spinner="⏳ Cargando y limpiando datos...")
def cargar_datos(ruta: str = "data/retail_sales.csv") -> pd.DataFrame:
    """
    Lee el CSV, lo limpia y devuelve un DataFrame listo para usar.
    Si no encuentra el CSV, genera datos de demostración.
    """

    # -- Leer el CSV ---------------------------------------------
    try:
        # parse_dates=["Date"] le dice a pandas que convierta la
        # columna "Date" a tipo datetime en vez de texto.
        # Esto nos permite luego extraer el mes y el año fácilmente.
        df = pd.read_csv(ruta, parse_dates=["Date"])
    except FileNotFoundError:
        # Si el archivo no existe, usamos datos sintéticos
        st.warning("⚠️ CSV no encontrado. Usando datos de demostración.")
        df = _generar_datos_demo()

    # -- Limpieza 1: eliminar "NaN?" en Category -------------------
    # El dataset tiene un problema: algunos valores de Category
    # son la cadena de texto "NaN?" (no un valor nulo real).
    # pandas no los detecta automáticamente como nulos.
    # Los detectamos manualmente y los eliminamos.
    #
    # astype(str) -> convierte todo a texto (por si hay nulos reales)
    # .str.strip() -> elimina espacios en blanco
    # .isin([...]) -> True si el valor está en la lista
    # ~ -> operador NOT (negación): nos quedamos con los que NO están en la lista
    df = df[~df["Category"].astype(str).str.strip().isin(
        ["NaN?", "nan", "NaN", ""]
    )]

    # También eliminamos nulos reales con dropna
    df = df.dropna(subset=["Category"])
```

```python
    # -- Limpieza 2: eliminar filas con valores críticos nulos ------
    # Si falta Sales, Profit, Quantity, Date o Region,
    # esa fila no sirve para el análisis -> la eliminamos
    df = df.dropna(subset=["Sales", "Profit", "Quantity", "Date", "Region"])

    # -- Limpieza 3: eliminar valores negativos --------------------
    # Ventas y cantidad negativas son datos anómalos (errores de carga)
    df = df[(df["Sales"] >= 0) & (df["Quantity"] >= 0)]

    # -- Limpieza 4: normalizar texto ------------------------------
    # .str.strip() elimina espacios
    # .str.title() capitaliza la primera letra: "electronics" -> "Electronics"
    df["Category"] = df["Category"].str.strip().str.title()
    df["Region"]   = df["Region"].str.strip().str.title()

    # -- Columnas derivadas ----------------------------------------
    # Estas columnas no están en el CSV original, las calculamos
    # una sola vez aquí para no repetir el cálculo en cada página.

    # dt.to_period("M") convierte la fecha a período mensual: "2023-01"
    # .astype(str) lo convierte a texto para usarlo como etiqueta
    df["Month"] = df["Date"].dt.to_period("M").astype(str)

    # dt.year extrae solo el año: 2023, 2024, etc.
    df["Year"]  = df["Date"].dt.year

    # Margen de ganancia en porcentaje: (Profit / Sales) * 100
    # replace(0, np.nan) evita división por cero
    df["Profit_Margin"] = (
        df["Profit"] / df["Sales"].replace(0, np.nan)
    ) * 100

    return df.reset_index(drop=True)


def _generar_datos_demo() -> pd.DataFrame:
    """
    Genera un DataFrame sintético con la misma estructura que el CSV de Kaggle.
    Se usa automáticamente si el CSV no está disponible.
    El guión bajo al inicio del nombre (_generar_datos_demo) es una
    convención de Python para indicar que esta función es "privada"
    (solo se usa dentro de este módulo).
    """
    np.random.seed(42)  # semilla para reproducibilidad (siempre los mismos datos)
    n = 1200            # número de registros a generar

    categorias = ["Electronics", "Furniture", "Office Supplies",
                  "Clothing", "Food & Beverages"]
    regiones   = ["North", "South", "East", "West", "Central"]
    fechas     = pd.date_range("2022-01-01", "2024-12-31", periods=n)

    cat = np.random.choice(categorias, n)
```

```
    reg = np.random.choice(regiones,   n)

    # Cada categoría tiene una venta base diferente para simular
    # diferencias reales de precio entre categorías
    base_ventas = {
        "Electronics":      800,
        "Furniture":        600,
        "Office Supplies":  200,
        "Clothing":         150,
        "Food & Beverages": 100,
    }

    # Añadimos ruido aleatorio (±40%) a las ventas base
    sales    = np.array([base_ventas[c] * (1 + np.random.randn() * 0.4) for c in
cat]).clip(10)
    quantity = np.random.randint(1, 50, n)
    profit   = sales * np.random.uniform(0.05, 0.35, n)

    return pd.DataFrame({
        "Date":     fechas,
        "Category": cat,
        "Sales":    sales.round(2),
        "Quantity": quantity,
        "Profit":   profit.round(2),
        "Region":   reg,
    })
```

¿Qué es un DataFrame?

Un DataFrame de pandas es como una hoja de Excel en Python: tiene filas y columnas, puedes filtrar, ordenar, calcular totales, etc. En este proyecto, todo el análisis se hace sobre DataFrames.

¿Qué es un decorador (@)?

El `@st.cache_data` es un decorador. Se coloca justo encima de una función y modifica su comportamiento. En este caso, le añade la funcionalidad de «guardar el resultado en memoria». No necesitas entender cómo funciona internamente — solo recuerda: pon `@st.cache_data` encima de funciones que leen archivos o hacen cálculos pesados.

Paso 5 — Crear `utils.py` — Funciones Compartidas

*`utils.py` es la librería interna del proyecto. Contiene funciones que **todas las páginas usan**: la barra lateral, los KPIs y todas las gráficas. Así evitamos copiar y pegar el mismo código en 4 archivos diferentes.*

Crea `retaildash/utils.py`:

```
import streamlit as st
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from data_loader import cargar_datos, REGION_COORDS
```

```python
# ==================================================================
#  SECCIÓN 1: FORMATEO DE NÚMEROS
# ==================================================================

def fmt_money(v: float) -> str:
    """
    Convierte un número a formato legible de dinero.
    Ejemplos:
      1_500_000 -> "$1.50M"
      45_000    -> "$45.0K"
      850       -> "$850"
    """
    if v >= 1_000_000:
        return f"${v/1_000_000:.2f}M"
    if v >= 1_000:
        return f"${v/1_000:.1f}K"
    return f"${v:.0f}"


# ==================================================================
#  SECCIÓN 2: BARRA LATERAL Y AUTENTICACIÓN
# ==================================================================

def sidebar_con_filtros() -> pd.DataFrame:
    """
    Función que hacen las cuatro páginas al inicio.
    Hace tres cosas:
      1. Verifica que el usuario esté logueado
      2. Muestra la sidebar con info de usuario, logout y filtros
      3. Devuelve el DataFrame con los filtros aplicados
    """

    # -- Guard de autenticación --------------------------------
    # Si alguien intenta entrar directamente a una página sin
    # haber hecho login, lo detenemos aquí.
    # .get("logged_in", False) devuelve False si la clave no existe
    if not st.session_state.get("logged_in", False):
        st.warning("⚠️ Debes iniciar sesión para ver esta página.")
        st.stop()  # detiene la ejecución -> la página queda en blanco

    # -- Contenido de la sidebar -------------------------------
    with st.sidebar:
        st.title("🛒 RetailDash")

        # f-string: muestra el nombre del usuario logueado
        # Los ** alrededor ponen el texto en negrita en Markdown
        st.write(f"**{st.session_state.nombre}**")
        st.caption(f"Rol: {st.session_state.rol}")
        st.divider()

        # Botón de logout
```

```
        if st.button("🚪 Cerrar sesión", use_container_width=True):
            st.session_state.logged_in = False
            st.rerun()  # recarga -> app.py detecta logged_in=False -> login

        st.divider()
        st.subheader("🔍 Filtros")

        # Cargamos los datos completos (desde caché, no re-lee el CSV)
        df_raw = cargar_datos()

        # -- Filtro de Categoría --------------------------------
        # unique() devuelve los valores únicos de la columna
        # sorted() los ordena alfabéticamente
        # ["Todas"] + [...] añade la opción "Todas" al principio
        categorias = ["Todas"] + sorted(df_raw["Category"].unique().tolist())
        cat_sel = st.selectbox("Categoría", categorias, key="cat_sel")
        # key="cat_sel" es obligatorio cuando el mismo widget
        # aparece en múltiples páginas. Sin key, Streamlit lanza error.

        # -- Filtro de Región -----------------------------------
        regiones = ["Todas"] + sorted(df_raw["Region"].unique().tolist())
        reg_sel = st.selectbox("Región", regiones, key="reg_sel")

        # -- Filtro de Año --------------------------------------
        # multiselect permite seleccionar varios años a la vez
        # default=años selecciona todos por defecto
        años = sorted(df_raw["Year"].unique().tolist())
        año_sel = st.multiselect("Año", años, default=años, key="año_sel")

    # -- Aplicar los filtros al DataFrame ----------------------
    # .copy() crea una copia para no modificar el DataFrame original
    df = df_raw.copy()

    if cat_sel != "Todas":
        df = df[df["Category"] == cat_sel]

    if reg_sel != "Todas":
        df = df[df["Region"] == reg_sel]

    if año_sel:
        # .isin([...]) filtra las filas cuyo año está en la lista
        df = df[df["Year"].isin(año_sel)]

    # Si los filtros dejan el DataFrame vacío, mostramos aviso
    if df.empty:
        st.warning("⚠️ No hay datos para los filtros seleccionados.")
        st.stop()

    return df


# ==================================================================
#  SECCIÓN 3: KPIs
```

```python
# ================================================================

def calcular_kpis(df: pd.DataFrame) -> dict:
    """Calcula los 6 indicadores clave y los devuelve en un diccionario."""
    total_ventas  = df["Sales"].sum()
    total_profit  = df["Profit"].sum()
    total_items   = int(df["Quantity"].sum())
    num_registros = len(df)
    margen_prom   = (total_profit / total_ventas * 100) if total_ventas else 0
    ticket_prom   = total_ventas / num_registros if num_registros else 0

    return {
        "ventas":    total_ventas,
        "profit":    total_profit,
        "items":     total_items,
        "registros": num_registros,
        "margen":    margen_prom,
        "ticket":    ticket_prom,
    }


def mostrar_kpis(df: pd.DataFrame) -> None:
    """
    Muestra los 6 KPIs como tarjetas con st.metric.
    st.columns(6) crea 6 columnas iguales, una por KPI.
    """
    kpis = calcular_kpis(df)

    k1, k2, k3, k4, k5, k6 = st.columns(6)
    k1.metric("💰 Ventas Totales",     fmt_money(kpis["ventas"]))
    k2.metric("📈 Ganancia Total",     fmt_money(kpis["profit"]))
    k3.metric("📦 Unidades Vendidas", f"{kpis['items']:,}")
    k4.metric("📋 Registros",          f"{kpis['registros']:,}")
    k5.metric("🎯 Margen Promedio",    f"{kpis['margen']:.1f}%")
    k6.metric("🧾 Ticket Promedio",    fmt_money(kpis["ticket"]))
    # :, en el f-string agrega separador de miles: 1500 -> 1,500
    # :.1f muestra 1 decimal: 23.456 -> 23.5


# ================================================================
#  SECCIÓN 4: GRÁFICAS CON PLOTLY
# ================================================================
#
# Todas las funciones de gráfica:
#   - Reciben un DataFrame df como parámetro
#   - Devuelven un objeto go.Figure de Plotly
#   - Son llamadas con st.plotly_chart(función(df), use_container_width=True)

def grafico_ventas_tiempo(df: pd.DataFrame) -> go.Figure:
    """Línea con área: ventas totales agrupadas por mes."""

    # groupby("Month") agrupa todas las filas con el mismo mes
    # ["Sales"].sum() suma las ventas de cada grupo
```

```
    ts = df.groupby("Month")["Sales"].sum().reset_index()

    # Convertimos el texto "2023-01" de vuelta a fecha para ordenar correctamente
    ts["Month_dt"] = pd.to_datetime(ts["Month"])
    ts = ts.sort_values("Month_dt")

    fig = go.Figure()
    fig.add_trace(go.Scatter(
        x=ts["Month_dt"],
        y=ts["Sales"],
        mode="lines+markers",              # línea con puntos
        line=dict(color="#FF4B4B", width=2.5),
        marker=dict(size=5),
        fill="tozeroy",                    # rellena el área bajo la línea
        fillcolor="rgba(255,75,75,0.10)",  # rojo semitransparente
        name="Ventas",
    ))
    fig.update_layout(title="📈 Ventas Mensuales", xaxis_title="", yaxis_title="USD")
    return fig


def grafico_profit_tiempo(df: pd.DataFrame) -> go.Figure:
    """Línea con área: ganancia total agrupada por mes."""
    ts = df.groupby("Month")["Profit"].sum().reset_index()
    ts["Month_dt"] = pd.to_datetime(ts["Month"])
    ts = ts.sort_values("Month_dt")

    fig = go.Figure()
    fig.add_trace(go.Scatter(
        x=ts["Month_dt"], y=ts["Profit"],
        mode="lines+markers",
        line=dict(color="#636EFA", width=2.5),
        marker=dict(size=5),
        fill="tozeroy",
        fillcolor="rgba(99,110,250,0.10)",
        name="Ganancia",
    ))
    fig.update_layout(title="📈 Ganancia Mensual", xaxis_title="", yaxis_title="USD")
    return fig


def grafico_ventas_anuales(df: pd.DataFrame) -> go.Figure:
    """Barras verticales: ventas totales por año."""
    anual = df.groupby("Year")["Sales"].sum().reset_index()

    # px.bar es una forma más simple de crear barras con Plotly Express
    # text_auto=".2s" muestra el valor sobre cada barra (ej: "45.0K")
    fig = px.bar(
        anual, x="Year", y="Sales",
        title="📊 Ventas por Año",
        text_auto=".2s",
        color="Sales",
        color_continuous_scale="Reds",
```

```
    )
    fig.update_layout(
        coloraxis_showscale=False,     # oculta la barra de color
        xaxis=dict(type="category"),   # trata los años como categorías, no números
    )
    return fig


def grafico_categoria_barras(df: pd.DataFrame) -> go.Figure:
    """Barras horizontales: ventas por categoría, de menor a mayor."""
    cat = df.groupby("Category")
["Sales"].sum().sort_values(ascending=True).reset_index()

    fig = go.Figure(go.Bar(
        x=cat["Sales"],
        y=cat["Category"],
        orientation="h",   # horizontal
        marker_color="#FF4B4B",
        text=[fmt_money(v) for v in cat["Sales"]],   # etiqueta en cada barra
        textposition="outside",
    ))
    fig.update_layout(title="🏷️ Ventas por Categoría", xaxis_title="")
    return fig


def grafico_margen_categoria(df: pd.DataFrame) -> go.Figure:
    """Barras verticales: margen de ganancia (%) por categoría."""

    # apply() aplica una función a cada grupo
    # La lambda calcula el margen: (suma profit / suma ventas) * 100
    # include_groups=False evita un warning de pandas moderno
    mg = df.groupby("Category").apply(
        lambda g: (g["Profit"].sum() / g["Sales"].sum() * 100) if g["Sales"].sum()
else 0,
        include_groups=False,
    ).reset_index(name="Margen")
    mg = mg.sort_values("Margen", ascending=False)

    # La barra más alta va en rojo, las demás en azul
    colores = ["#FF4B4B" if v == mg["Margen"].max() else "#636EFA" for v in
mg["Margen"]]

    fig = go.Figure(go.Bar(
        x=mg["Category"], y=mg["Margen"],
        marker_color=colores,
        text=[f"{v:.1f}%" for v in mg["Margen"]],
        textposition="outside",
    ))
    fig.update_layout(title="💹 Margen de Ganancia por Categoría (%)",
yaxis_title="%")
    return fig
```

```python
def grafico_region_pie(df: pd.DataFrame) -> go.Figure:
    """Donut: participación porcentual de ventas por región."""
    reg = df.groupby("Region")["Sales"].sum().reset_index()

    fig = go.Figure(go.Pie(
        labels=reg["Region"],
        values=reg["Sales"],
        hole=0.45,   # el "hueco" del donut (0 = pastel, 1 = anillo vacío)
    ))
    fig.update_layout(title="🌎 Distribución de Ventas por Región")
    return fig


def grafico_region_barras(df: pd.DataFrame) -> go.Figure:
    """Barras agrupadas: ventas y ganancia por región."""
    reg = df.groupby("Region").agg(
        Ventas=("Sales", "sum"),
        Ganancia=("Profit", "sum"),
    ).reset_index().sort_values("Ventas", ascending=False)

    fig = go.Figure()
    fig.add_trace(go.Bar(name="Ventas",   x=reg["Region"], y=reg["Ventas"],
marker_color="#FF4B4B"))
    fig.add_trace(go.Bar(name="Ganancia", x=reg["Region"], y=reg["Ganancia"],
marker_color="#636EFA"))
    fig.update_layout(barmode="group", title="📊 Ventas y Ganancia por Región")
    return fig


def grafico_scatter_ventas_profit(df: pd.DataFrame) -> go.Figure:
    """
    Scatter plot: cada punto es una transacción.
    Eje X = ventas, Eje Y = ganancia.
    Color = categoría, tamaño = cantidad de unidades.
    trendline="ols" añade una línea de tendencia (regresión lineal).
    """
    fig = px.scatter(
        df.sample(min(len(df), 600), random_state=1),  # máximo 600 puntos para
rendimiento
        x="Sales", y="Profit",
        color="Category",
        size="Quantity",
        opacity=0.7,
        title="🔵 Ventas vs Ganancia",
        trendline="ols",
    )
    return fig


def grafico_categoria_treemap(df: pd.DataFrame) -> go.Figure:
    """
    Treemap: rectángulos anidados Categoría -> Región.
    El área de cada rectángulo es proporcional a las ventas.
```

```
    Útil para ver la composición de ventas de un vistazo.
    """
    agg = df.groupby(["Category", "Region"]).agg(
        Sales=("Sales", "sum")
    ).reset_index()

    fig = px.treemap(
        agg,
        path=["Category", "Region"],   # jerarquía: primero categoría, luego región
        values="Sales",
        title="📊 Composición de Ventas: Categoría -> Región",
        color="Sales",
        color_continuous_scale="Reds",
    )
    fig.update_layout(coloraxis_showscale=False)
    return fig


def grafico_mapa_regiones(df: pd.DataFrame) -> go.Figure:
    """
    Mapa de burbujas sobre EE.UU.
    Cada burbuja representa una región. El tamaño es proporcional a las ventas.
    """
    reg = df.groupby("Region").agg(
        Sales=("Sales", "sum"),
        Profit=("Profit", "sum"),
        Registros=("Sales", "count"),
    ).reset_index()

    # Añadimos las coordenadas desde el diccionario REGION_COORDS
    reg["lat"] = reg["Region"].map(lambda r: REGION_COORDS.get(r, {}).get("lat", 0))
    reg["lon"] = reg["Region"].map(lambda r: REGION_COORDS.get(r, {}).get("lon", 0))

    # Texto que aparece al pasar el mouse por encima de cada burbuja
    reg["texto_hover"] = reg.apply(
        lambda r: (
            f"<b>{r['Region']}</b><br>"
            f"Ventas: {fmt_money(r['Sales'])}<br>"
            f"Ganancia: {fmt_money(r['Profit'])}<br>"
            f"Registros: {r['Registros']}"
        ),
        axis=1,
    )

    fig = go.Figure(go.Scattergeo(
        lat=reg["lat"],
        lon=reg["lon"],
        text=reg["texto_hover"],
        hoverinfo="text",
        mode="markers",
        marker=dict(
            # Tamaño proporcional a las ventas: la región con más ventas
            # tendrá burbuja de 60px, la de menos ventas de 15px
```

```
            size=(reg["Sales"] / reg["Sales"].max() * 45 + 15),
            color=reg["Sales"],
            colorscale="Reds",
            showscale=True,
            colorbar=dict(title="Ventas USD"),
            line=dict(color="white", width=1),
            opacity=0.85,
        ),
    ))

    # scope="usa" limita el mapa a los Estados Unidos
    fig.update_geos(scope="usa")
    fig.update_layout(title="🗺 Mapa de Ventas por Región (EE.UU.)", height=480)
    return fig
```

Paso 6 — Crear las páginas del dashboard — carpeta `pages/`

Cada página sigue el mismo patrón. Las primeras dos líneas siempre son iguales:

```
import streamlit as st
from utils import sidebar_con_filtros, ...  # importar las funciones necesarias

df = sidebar_con_filtros()  # inicializa la sidebar Y devuelve datos filtrados
```

Después de esas dos líneas, cada página hace lo suyo.

Página 1 — `pages/1_Resumen.py`:

```
import streamlit as st
from utils import (
    sidebar_con_filtros,
    mostrar_kpis,
    grafico_ventas_tiempo,
    grafico_profit_tiempo,
    grafico_ventas_anuales,
)

df = sidebar_con_filtros()

st.title("📊 Resumen Ejecutivo")
st.caption(f"Bienvenido/a, **{st.session_state.nombre}** · Rol:
{st.session_state.rol}")
st.divider()

st.subheader("Indicadores Clave de Rendimiento")
mostrar_kpis(df)
st.divider()

st.subheader("Tendencias Temporales")
col1, col2 = st.columns(2)
with col1:
    st.plotly_chart(grafico_ventas_tiempo(df),  use_container_width=True)
with col2:
```

```
    st.plotly_chart(grafico_profit_tiempo(df),  use_container_width=True)

st.plotly_chart(grafico_ventas_anuales(df), use_container_width=True)

st.caption("💡 Usa los filtros de la barra lateral para segmentar los datos.")
```

Página 2 — `pages/2_Ventas.py`:

```
import streamlit as st
from utils import (
    sidebar_con_filtros,
    grafico_categoria_barras,
    grafico_margen_categoria,
    grafico_region_pie,
    grafico_region_barras,
    grafico_scatter_ventas_profit,
    grafico_categoria_treemap,
)

df = sidebar_con_filtros()

st.title("💰 Análisis de Ventas")
st.caption("Desglose por categoría, región y relación ventas-ganancia.")
st.divider()

st.subheader("Por Categoría")
col1, col2 = st.columns(2)
with col1:
    st.plotly_chart(grafico_categoria_barras(df),  use_container_width=True)
with col2:
    st.plotly_chart(grafico_margen_categoria(df),  use_container_width=True)

st.divider()

st.subheader("Por Región")
col3, col4 = st.columns(2)
with col3:
    st.plotly_chart(grafico_region_pie(df),    use_container_width=True)
with col4:
    st.plotly_chart(grafico_region_barras(df), use_container_width=True)

st.divider()

st.subheader("Composición Categoría -> Región")
st.plotly_chart(grafico_categoria_treemap(df), use_container_width=True)

st.divider()

st.subheader("Relación Ventas vs Ganancia")
st.info("Cada punto es una transacción. El tamaño indica las unidades vendidas.")
st.plotly_chart(grafico_scatter_ventas_profit(df), use_container_width=True)
```

Página 3 — `pages/3_Geografico.py`:

```python
import streamlit as st
from utils import sidebar_con_filtros, grafico_mapa_regiones, fmt_money

df = sidebar_con_filtros()

st.title("🗺️ Mapa Geográfico de Ventas")
st.caption("Distribución espacial de ventas por región en EE.UU.")
st.divider()

st.plotly_chart(grafico_mapa_regiones(df), use_container_width=True)
st.divider()

# Tabla resumen por región
st.subheader("Resumen por Región")

reg_resumen = (
    df.groupby("Region")
    .agg(
        Ventas=("Sales",    "sum"),
        Ganancia=("Profit", "sum"),
        Unidades=("Quantity","sum"),
        Registros=("Sales", "count"),
    )
    .reset_index()
    .sort_values("Ventas", ascending=False)
)

reg_resumen["Margen (%)"] = (
    reg_resumen["Ganancia"] / reg_resumen["Ventas"] * 100
).round(1)

# Formatear las columnas de dinero
reg_resumen["Ventas"]   = reg_resumen["Ventas"].apply(fmt_money)
reg_resumen["Ganancia"] = reg_resumen["Ganancia"].apply(fmt_money)

st.dataframe(
    reg_resumen,
    use_container_width=True,
    hide_index=True,
    column_config={
        "Margen (%)": st.column_config.ProgressColumn(
            "Margen (%)",
            format="%.1f%%",
            min_value=0,
            max_value=100,
        ),
    },
)
```

Página 4 — `pages/4_Datos.py`:

```python
import streamlit as st
from utils import sidebar_con_filtros, fmt_money, calcular_kpis
```

```
df = sidebar_con_filtros()

st.title("📂 Explorador de Datos")
st.caption("Inspecciona, filtra y descarga el dataset.")
st.divider()

# Métricas rápidas
kpis = calcular_kpis(df)
c1, c2, c3 = st.columns(3)
c1.metric("Total de registros",     f"{kpis['registros']:,}")
c2.metric("Ventas en el periodo",   fmt_money(kpis["ventas"]))
c3.metric("Categorías distintas",   df["Category"].nunique())

st.divider()
st.subheader("Tabla de Datos")

# Controles de búsqueda y orden
col_buscar, col_orden, col_dir = st.columns([2, 1, 1])
with col_buscar:
    busqueda = st.text_input("🔎 Buscar en Categoría o Región", placeholder="Ej:
Electronics")
with col_orden:
    columna_orden = st.selectbox("Ordenar por", ["Date", "Sales", "Profit",
"Quantity"])
with col_dir:
    direccion = st.radio("Dirección", ["Descendente", "Ascendente"], horizontal=True)

# Aplicar búsqueda
df_vista = df.copy()
if busqueda:
    mask = (
        df_vista["Category"].str.contains(busqueda, case=False, na=False) |
        df_vista["Region"].str.contains(busqueda,   case=False, na=False)
    )
    df_vista = df_vista[mask]

# Aplicar orden
df_vista = df_vista.sort_values(
    columna_orden,
    ascending=(direccion == "Ascendente")
)

st.caption(f"Mostrando {min(500, len(df_vista)):,} de {len(df_vista):,} registros.")

st.dataframe(
    df_vista.head(500).style.format({
        "Sales":         "${:,.2f}",
        "Profit":        "${:,.2f}",
        "Profit_Margin": "{:.1f}%",
    }),
    use_container_width=True,
    height=420,
)
```

```
st.divider()
st.subheader("⬇ Descargar Datos")

col_dl1, col_dl2 = st.columns(2)
with col_dl1:
    csv = df_vista.to_csv(index=False).encode("utf-8")
    st.download_button(
        label="Descargar CSV filtrado",
        data=csv,
        file_name="retaildash_filtrado.csv",
        mime="text/csv",
        use_container_width=True,
    )
with col_dl2:
    resumen = (
        df.groupby("Category")
        .agg(Ventas=("Sales","sum"), Ganancia=("Profit","sum"),
Unidades=("Quantity","sum"))
        .reset_index()
    )
    csv_res = resumen.to_csv(index=False).encode("utf-8")
    st.download_button(
        label="Descargar resumen por categoría",
        data=csv_res,
        file_name="retaildash_resumen.csv",
        mime="text/csv",
        use_container_width=True,
    )
```

Paso 7 — Ejecutar y probar el proyecto

Ya tienes todos los archivos. Ahora vamos a ejecutar la app y verificar que todo funciona.

Paso 1 — Asegúrate de estar en la carpeta correcta:

```
cd retaildash
```

Paso 2 — Ejecuta la app:

```
streamlit run app.py
```

Deberías ver en la terminal algo como:

```
You can now view your Streamlit app in your browser.
Local URL: http://localhost:8501
```

Y el navegador se abrirá automáticamente.

Paso 3 — Prueba el flujo completo:

1. *Ingresa con `admin` / `admin123` -> deberías ver el dashboard*
2. *Navega entre las 4 páginas usando la barra lateral*
3. *Aplica filtros de categoría y región*

4. *Haz clic en «Cerrar sesión» -> deberías volver al login*
5. *Intenta entrar con credenciales incorrectas -> debería aparecer error en rojo*

Credenciales de prueba:

Usuario	***Contraseña***	***Rol***
`admin`	`admin123`	*Admin*
`ana`	`ana2024`	*Analista*
`carlos`	`carlos2024`	*Viewer*

Errores comunes y soluciones:

Error	***Solución***
`ModuleNotFoundError: No module named 'plotly'`	*Ejecuta:* `pip install plotly`
`FileNotFoundError: data/retail_sales.csv`	*La app usa datos de demo automáticamente. Para usar el CSV real, descárgalo de Kaggle y colócalo en* `data/`
`StreamlitDuplicateWidgetId`	*Verifica que todos los widgets de la sidebar tengan* `key=` *único:* `key="cat_sel"`, `key="reg_sel"`, *etc.*
`set_page_config() can only be called once`	*Solo* `app.py` *debe llamar a* `st.set_page_config()`. *Elimínalo de cualquier otro archivo*
La app recarga constantemente sin parar	*Revisa que* `st.rerun()` *solo se llame dentro de un* `if` *condicional, nunca en el flujo principal*

Paso 4 — Para detener la app:

Presiona `Ctrl + C` *en la terminal.*

7. Resumen Final — ¿Qué aprendiste?

En este proyecto aplicaste **todos** los conceptos del curso en una aplicación real:

Módulo	**Concepto aplicado**	**Dónde**
M1	`set_page_config`, scripting flow, widgets básicos	`app.py`, todas las páginas
M2	`session_state`, `@st.cache_data`, `st.columns`	`app.py`, `auth.py`, `data_loader.py`
M3	`st.plotly_chart`, `st.dataframe`, `st.metric`	`utils.py`, todas las páginas
M4	`st.form`, `st.navigation`, `st.rerun()`, `type="password"`	`app.py`, `auth.py`

Arquitectura del proyecto:

Archivo	Responsabilidad
`app.py`	Director de orquesta: decide qué mostrar según `session_state`
`auth.py`	Login: valida credenciales y actualiza `session_state`
`data_loader.py`	Datos: lee el CSV, lo limpia y lo cachea con `@st.cache_data`
`utils.py`	Librería interna: sidebar, KPIs y todas las gráficas
`pages/*.py`	Una responsabilidad por página: importan de `utils.py` y componen la vista